

Perfect. Since you specifically said **"for this prompt only, don't go too deep"**, I'll keep it **medium depth**, focusing on **study + interview + practical coding** rather than internal implementation.

1. Understand File Path vs File Name

Many beginners confuse these two.

File Name

The actual name of the file.

Example

text

students.txt

,

or

text

data.csv

,

File Path

The location of the file.

Relative Path

text

data/students.txt

,

Java searches relative to your project's working directory.

Absolute Path

Windows

text

C:\Users\Siraj\Documents\students.txt

`

Linux/Mac

text

/home/siraj/students.txt

`

Example

java

Path path = Path.of("data/students.txt");

`

Here

* File name → students.txt`

* File path → data/students.txt`

Common Mistake

✗

java

Path.of("C:\\Users\\Siraj\\Desktop\\data.txt");

`

Works only on **your computer**.

✓ Better

java

Path.of("data/data.txt");

`

Interview Questions

Difference between file path and file name?

Answer

- * File name identifies the file.

- * File path identifies the file's location.

2. Read Text Files

Java provides several ways.

Method 1 — Files.readAllLines()

Best for **small files**.

Example

Suppose students.txt`

text

Ali

Ahmed

Sara

`

java

```
import java.nio.file.*;
```

```
import java.util.*;
```

```
List lines =
```

```
Files.readAllLines(Path.of("students.txt"));
```

```
for(String line : lines){
```

```
System.out.println(line);
```

```
}
```

`

Output

`

Ali

Ahmed

Sara

`

Method 2 — BufferedReader`

Better for **large files**.

java

BufferedReader reader =

Files.newBufferedReader(Path.of("students.txt"));

String line;

while((line = reader.readLine()) != null){

System.out.println(line);

}

reader.close();

`

When to Use?

| Method | Best For |

| ----- | ----- |

| Files.readAllLines()`| Small files |

| BufferedReader`| Large files |

Interview Question

Why use BufferedReader`for large files?

Answer

Because it reads one line at a time instead of loading the whole file into memory.

3. Write Text Files

Method 1 — Files.write()

java

List names =

List.of("Ali","Ahmed","Sara");

Files.write(Path.of("students.txt"), names);

、

File becomes

text

Ali

Ahmed

Sara

、

Method 2 — PrintWriter

java

PrintWriter writer =

new PrintWriter("students.txt");

writer.println("Ali");

writer.println("Ahmed");

writer.println("Sara");

writer.close();

、

Interview Question

Which is simpler for writing a few lines?

Answer

Files.write() `

4. Use Try-With-Resources

One of Java's best features.

Instead of manually closing resources,

Bad

java

BufferedReader reader =

Files.newBufferedReader(Path.of("students.txt"));

reader.close();

`

If an exception occurs before `close()`, the resource may remain open.

Good

java

try (BufferedReader reader =

Files.newBufferedReader(Path.of("students.txt"))) {

String line;

while((line = reader.readLine()) != null){

System.out.println(line);

}

}

`

Java automatically closes the resource.

Works with

- * BufferedReader
- * PrintWriter
- * Scanner
- * FileInputStream
- * FileOutputStream

Interview Question

Why use try-with-resources?

Answer

It automatically closes resources, even if an exception occurs.

5. Byte Stream vs Character Stream

Byte Stream

Processes raw bytes.

Classes

↳ text

FileInputStream

FileOutputStream

、

Used for

- * Images
- * PDFs
- * Videos
- * Audio

Character Stream

Processes characters.

Classes

↳ text

FileReader

FileWriter

BufferedReader

PrintWriter

、

Used for

* Text

* CSV

* JSON

* XML

Interview Summary

| Byte Stream | Character Stream |

| ----- | ----- |

| Bytes | Characters |

| Binary files | Text files |

| FileInputStream | FileReader |

| FileOutputStream | FileWriter |

6. Use CSV-like Text Format

CSV =

↳ text

Comma Separated Values

、

Example

text

Ali,22,CS

Ahmed,20,SE

Sara,21,IT

、

Reading

java

```
String line = "Ali,22,CS";
```

```
String[] parts = line.split(",");
```

```
System.out.println(parts[0]);
```

```
System.out.println(parts[1]);
```

```
System.out.println(parts[2]);
```

、

Output

、

Ali

22

CS

、

Very common for beginner persistence.

Interview Question

Why use CSV?

Simple.

Human readable.

Easy to parse.

7. Java Serialization Concept

Serialization means

> Converting an object into bytes so it can be stored or transferred.

Example

text

Student Object

↓

Bytes

↓

student.ser

,

Deserialization

text

Bytes

↓

Student Object

,

To serialize

java

class Student implements Serializable{

}

,

Uses

* Learning

* Sending objects over network

- * Saving temporary object state

Limitations

- * Java-specific

- * Not human readable

- * Version compatibility issues

- * Not suitable for databases or APIs

Interview Question

What is serialization?

Answer

Converting an object into bytes for storage or transmission.

8. Don't Use Serialization as Default Storage

Modern applications prefer

- * Database

- * JSON

- * XML

instead of Java Serialization.

Reason

- * More portable

- * Easier to debug

- * Language independent

Serialization is good for learning but is rarely the primary storage format in production.

Interview Question

Should serialization be used for production storage?

Answer

Generally no. JSON, databases, or other portable formats are preferred.

9. Handle Missing Files Gracefully

Bad

```
java
```

```
Files.readAllLines(Path.of("students.txt"));
```

`

If the file doesn't exist

↓

Program crashes.

Good

```
java
```

```
try {
```

```
List lines =
```

```
Files.readAllLines(Path.of("students.txt"));
```

```
} catch (IOException e){
```

```
System.out.println("File not found.");
```

```
}
```

`

Invalid Content Example

Suppose file contains

text

Ali

Twenty

CS

、

Trying

java

int age =

Integer.parseInt("Twenty");

、

Throws

text

NumberFormatException

、

Handle it

java

try{

int age =

Integer.parseInt(value);

}catch(NumberFormatException e){

System.out.println("Invalid age.");

}

、

Best Practice

Separate

File Reading

text

Read file

↓

Convert into objects

、

from

Business Logic

text

Calculate GPA

↓

Generate Report

↓

Find Top Student

、

Don't mix everything in one method.

Common Mistake

Hardcoding

java

"C:\\Users\\Siraj\\Desktop\\students.txt"

、

Fails on another computer.

Better

java

Path.of("data/students.txt")

、

Interview Insight: IOException

IOException is a **checked exception**.

That means Java forces you to either:

```
java
```

```
try-catch
```

```
,
```

```
or
```

```
java
```

```
throws IOException
```

```
,
```

Example

```
java
```

```
public void readFile() throws IOException{
```

```
}
```

```
,
```

OR

```
java
```

```
try{
```

```
}catch(IOException e){
```

```
}
```

```
,
```

```
---
```

Why Checked?

Because file operations can fail due to:

- * Missing file

- * No permission

- * Disk errors

- * File locked by another program

Java forces you to handle these possibilities.

```
---
```

Resource Cleanup

Old Way

```
java
```

```
reader.close();
```

```
,
```

Easy to forget.

Modern Way

```
java
```

```
try(BufferedReader reader = ...){
```

```
}
```

```
,
```

Automatic cleanup.

Interview Questions (With Answers)

Q1. Difference between file path and file name?

Answer: File name identifies the file, while the file path specifies its location.

Q2. Files.readAllLines() vs BufferedReader?

Answer: `readAllLines()` loads the entire file into memory (good for small files), whereas `BufferedReader` reads one line at a time (better for large files).

Q3. Why use try-with-resources?

Answer: It automatically closes resources even if an exception occurs.

Q4. Byte stream vs character stream?

Answer: Byte streams handle binary data (images, PDFs), while character streams handle text data.

Q5. What is serialization?

Answer: Converting an object into bytes so it can be stored or transmitted.

Q6. Why isn't serialization preferred for production storage?

Answer: It's Java-specific, not human-readable, and has version compatibility issues. JSON or databases are generally better choices.

Q7. Why is `IOException` checked?

Answer: Because file operations can fail for many reasons, Java requires you to handle or declare the exception.

Q8. How do you safely close file resources?

Answer: Use try-with-resources.

Q9. How should you handle a missing file?

Answer: Catch `IOException` and handle it gracefully instead of letting the program crash.

Q10. Why separate file parsing from business logic?

Answer: It makes the code cleaner, easier to test, maintain, and reuse.

One-Line Interview Revision

* **File Name** → Name of the file (`students.txt`).

* **File Path** → Location of the file (`data/students.txt`).

* **`Files.readAllLines()`** → Reads the whole file; suitable for small files.

* **`BufferedReader`** → Reads line by line; suitable for large files.

* **`Files.write()`** → Simple way to write text files.

* **`PrintWriter`** → Writes formatted text to a file.

* **Try-with-resources** → Automatically closes resources.

- * **Byte Stream** → For binary files.
- * **Character Stream** → For text files.
- * **CSV** → Simple comma-separated text format.
- * **Serialization** → Converts an object to bytes.
- * **Production Storage** → Prefer JSON or databases over Java serialization.
- * **IOException** → Checked exception for file I/O operations.
- * **Best Practice** → Keep file parsing separate from business logic.
- * **Avoid** → Hardcoded absolute file paths.